Enhancement Plan

Sentiment Analysis Web Application

Name of Student	BITS ID	Contribution
K ROMA PAI	2024aa05965@wilp.bits-pilani.ac.in	100%
JITESH GUPTA	2024ab05020@wilp.bits-pilani.ac.in	100%
KOTHA AMITABH	2024ab05195@wilp.bits-pilani.ac.in	100%
JOHIT GARG	2024aa05907@wilp.bits-pilani.ac.in	100%
KARTHIK REDDY S	2024ab05330@wilp.bits-pilani.ac.in	100%

1. System Architecture Overview

- **Core design**: Flask web application using TF-IDF vectorization with a soft voting ensemble of linear classifiers.
- **Classification scope**: Three sentiment classes (Positive, Negative, Neutral) with probabilistic outputs.
- **Resource efficiency**: Approximately 6 MB model size enabling CPU-only inference under 100 milliseconds.
- **Deployment strength**: Optimized for edge environments without cloud dependency or specialized hardware.

The current system establishes a deliberately lightweight architecture that prioritizes operational efficiency over maximal accuracy. By combining sparse TF-IDF representations with linear classifiers in a soft voting ensemble, it achieves rapid inference while maintaining respectable performance on standard sentiment tasks. This minimal design delivers genuine value for resource constrained deployments such as IoT edge devices, offline mobile applications, or embedded systems where model footprint and start up time critically impact user experience. The compact 6 MB artifact loads quickly and processes inputs without specialized hardware, creating a practical foundation that balances capability with deploy ability. Strategic enhancements can substantially improve real world robustness while preserving this core efficiency advantage.

2. Linguistic Representation Enhancements

2.1 Context-Aware Feature Engineering

- **N-gram expansion**: Add bi-grams and tri-grams to capture phrasal sentiment patterns
- **TF scaling**: Apply sublinear term frequency transformation to moderate high-frequency term influence
- **Vocabulary control**: Restrict feature space to 10,000–20,000 terms for improved generalization
- **Character-level features**: Include 3–5-character n-grams to handle spelling variations.

Expanding beyond single word representations transforms the model's ability to detect contextual sentiment inversion. The phrase *"not good"* correctly triggers **negative** classification whereas unigram models see only the **positive** word *"good"*. Similarly, tri-grams capture intensification patterns like *"very impressive"* and qualified sentiment like *"barely acceptable"* that define nuanced human expression. Character n-grams provide unexpected resilience against misspellings. The misspelled word *"awesomme"* shares sufficient character sub-sequences with *"awesome"* to trigger appropriate sentiment signals without explicit dictionary lookups.

2.2 Negation Scope Modeling

- **Scope-aware transformation**: Prefix negation markers to subsequent tokens within clause boundaries.
- **Pragmatic handling**: Special rules for double negation constructions like "not bad" indicating mild positivity.
- **Preprocessing integration**: Apply transformations before vectorization to maintain classifier compatibility.

Negation handling remains challenging for bag of words models that discard syntactic structure yet depend on it for semantic inversion. The scope aware negation tagging transforms *"not good experience"* into *"not_good not_experience" during* preprocessing. This preserves negation context through lexical fusion while maintaining compatibility with linear models. This requires either lexicon-based rules or training data with such constructions. Crucially, this enhancement operates entirely within the preprocessing pipeline requiring zero modifications to the classifier architecture while delivering disproportionate accuracy gains on negated constructions common in authentic user feedback.

2.3 Informal Language Normalization

- **Orthographic compression**: Reduce repeated characters like "goooood" to "good" while preserving intensity signals
- **Slang mapping**: Convert informal expressions like "salty" to "angry" and "fire" to "excellent"
- **Emoji semantics**: Transform pictographic symbols into sentiment bearing tokens using established mappings
- **Pipeline integration**: Apply normalization before vectorization to enrich feature representation

Contemporary digital communication operates under distinct linguistic conventions that systematically diverge from formal text corpora. User inputs have expressive tone like *"sooo happy"*, abbreviations like *"omg"*, and ***emoji sequences*** carrying significant affective weight. A curated slang dictionary bridges generational and subcultural language gaps. Emoji processing converts pictographs into sentiment tokens such as converting a laughing emoji to *"laughter_positive"* and an angry face to *"anger_negative"*. These **transformations** occur **before vectorization** ensuring the TF-IDF representation captures sentiment signals embedded in informal language while maintaining computational efficiency.

3. Prediction Confidence and Interpretability

- **Probability calibration**: Apply Platt scaling to align predicted probabilities with observed frequencies
- **Confidence tiering**: Implement visual indicators for low confidence predictions below 65 percent and high confidence above 90 percent
- **Transparent display**: Show full probability distribution rather than single class output
- **Trust building**: Transform opaque predictions into interpretable analytical judgments

The soft voting ensemble inherently produces probability distributions across sentiment classes whereas the raw classifiers outputs often exhibit miscalibration. Predictions falling below a configurable threshold trigger visual indicators prompting user verification while high confidence predictions receive subtle reinforcement cues. Users immediately perceive ambiguous cases were sentiment signals conflict across modalities. This tiered transparency transforms the system from an opaque classifier into a collaborative analytical tool where users understand both the prediction and its reliability.

4. Adaptive Learning Through Structured Feedback

- **Closed loop design**: Integrate one click correction prompts after each prediction asking *"Was this accurate?"*.
- **Lightweight storage**: Store corrections in embedded SQLite database with input text labels and timestamps.
- **Pattern discovery**: Query feedback repository to identify systematic failure modes and emerging linguistic trends.
- **Efficient retraining**: Leverage compact model size for rapid retraining cycles under two minutes on consumer hardware.

Instead of remaining a static model after deployment, the system can evolve through user interaction. Verification prompts allow users to correct misclassifications, and these inputs—along with predictions and metadata—are stored in a lightweight SQLite database for analysis.

Given the model's compact size and linear architecture, retraining can be performed quickly on modest hardware, enabling regular update cycles without heavy infrastructure. This feedback-driven loop supports gradual adaptation to domain-specific language patterns with minimal operational overhead.

5. Multilingual Expansion Strategy

- **Language detection**: Integrate compact detection library for automatic routing of inputs.
- **Parallel pipelines**: Maintain language specific TF-IDF vectorizers and classifiers per supported language.
- **Configuration consistency**: Standardize n gram ranges and max features across languages with vocabulary adjustments.
- **Phased rollout**: Begin with high value languages before considering cross lingual embedding approaches.

The current English only implementation establishes a foundation scalable through a pragmatic multilingual strategy that avoids premature architectural complexity. Each supported language maintains its own **TF-IDF vectorizer** and classifier ensemble trained on sentiment annotated corpora in that language. **Future iterations** could incorporate multilingual embeddings for true cross lingual transfer but the language specific pipeline approach delivers production ready multilingual support with minimal architectural changes.

6. Architectural Refinements for Maintainability

- **Modular decomposition**: Separate concerns into dedicated modules for preprocessing classification feedback and utilities.
- **Structured logging**: Capture prediction requests latencies errors and class distributions at configurable verbosity.
- **Robust validation**: Implement Unicode normalization length enforcement and injection sanitization.
- **Testability enhancement**: Enable isolated unit testing of preprocessing transformations independent of model behavior.

The application follows a modular design to ensure maintainability and scalability. Separate modules handle preprocessing, model prediction, and user feedback management. This separation enables independent testing of components and allows future upgrades—such as replacing the classifier—without affecting the UI.

Comprehensive logging supports performance monitoring and model evaluation. Tracking sentiment distribution helps detect domain drift, while latency monitoring identifies performance issues early. Error logging assists in refining preprocessing logic. Input validation, including Unicode normalization, length restrictions, and sanitization, enhances overall system robustness and security.

7. User Experience and Analytical Capabilities

- **Visual probability display**: Render full class distribution via horizontal bar charts conveying prediction certainty
- **Guided input**: Provide real time character counting and example suggestions demonstrating optimal input characteristics
- **Analytics dashboard**: Generate sentiment distribution histograms misclassification patterns and feature weight visualizations
- **Actionable insights**: Transform raw predictions into operational intelligence through aggregated trend analysis

The interface goes beyond simple sentiment labels by displaying probability distributions to show model confidence and highlight ambiguous cases. Real-time character counting and guided input suggestions enhance usability and improve input quality.

An integrated analytics view summarizes overall sentiment trends, identifies frequently misclassified phrases, and visualizes key feature weights for transparency. These insights are generated efficiently using lightweight SQLite aggregation, ensuring minimal computational overhead.

8. Resource Optimization Considerations

- **Feature selection**: Apply chi square statistical testing to retain only top 30 to 40 percent most discriminative terms
- **Serialization compression**: Utilize joblib compression level three for additional 15 to 20 percent size reduction
- **Memory footprint**: Reduce inference time memory consumption critical for constrained edge devices
- **Latency preservation**: Maintain sub 100 millisecond inference despite optimizations through sparse representation retention

Although the model is already compact at 6 MB, further optimization can improve deployment flexibility in constrained environments. Applying chi-square feature selection can reduce vocabulary size by 30–40% while maintaining accuracy by removing less informative features. Joblib compression can further shrink the serialized model by 15–20% with minimal loading overhead.

These optimizations lower memory usage and support deployment on resource-limited devices such as Raspberry Pi or edge systems. Importantly, they preserve fast CPU-only inference (under 100 ms), and reduced dimensionality can even improve prediction speed by minimizing sparse matrix computations.

9. Strategic Roadmap for Advanced Capabilities

- **Phased migration path**: Initially deploy BERT embeddings as optional feature set alongside TF-IDF for comparative evaluation
- **Accuracy cost analysis**: Quantify precision gains against computational overhead before full architectural transition
- **Domain specific justification**: Reserve transformer adoption for applications requiring nuanced contextual understanding
- **Hybrid architecture**: Maintain linear ensemble as default while offering advanced models as premium capability

The current linear ensemble balances accuracy and efficiency effectively. For tasks requiring deeper contextual understanding, such as sarcasm detection, transformer-based models like BERT could be introduced alongside TF-IDF for comparison before full adoption.

This incremental approach ensures upgrades are guided by measurable benefits rather than theoretical improvements, preserving the system's lightweight and efficient design.

10. Conclusion

- **Foundation strength**: 6 MB footprint and linear architecture deliver genuine edge deployment capability
- **Enhancement philosophy**: Strategic improvements augment linguistic sophistication without compromising core efficiency
- **Adaptive capability**: Closed loop learning transforms static model into evolving analytical asset through user feedback
- **Production readiness**: Balance between laboratory grade accuracy and real world deployability defines true operational value

The current sentiment analysis system provides a strong foundation with efficient performance, simple deployment, and solid baseline accuracy. Its compact linear architecture enables fast inference without specialized hardware, making it suitable for edge environments.

Enhancements in contextual feature engineering, structured feedback integration, and multilingual support can strengthen real-world robustness while maintaining efficiency. These improvements help bridge the gap between research-grade models and practical production systems that must handle diverse, evolving user language with limited resources.